

6.5 数组的查找与排序



6.5 数组的应用-数组元素的线性查找

线性查找（假设数组元素是无序排列的）

思路：从第一个元素开始依次找，直到找到或全部找完但没有找到。**时间复杂度 $O(n)$**

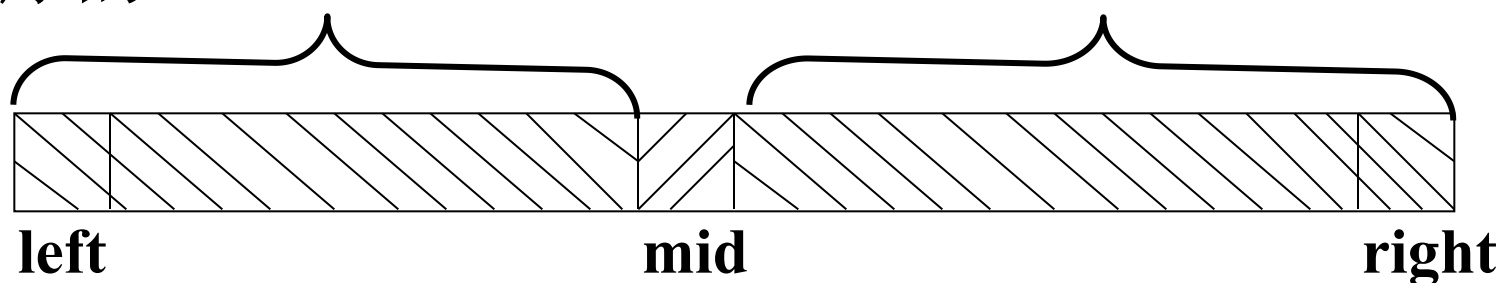
```
int findEle(int a[], int n, int x)
{
    int i;
    for (i=0; i<n && a[i] != x; i++)
        ;
    if (i >=n)
        return -1;
    else    return i;//返回存储位置
}
```

如何快速查找？

- 回想生活中我们如何快速查找东西的？
- 面对厚厚的字典，我们借助索引。索引技术广泛用在数据库和文件管理领域。
- 但是索引方法需要我们费心维护索引表和内容数组之间的一致性关系。这里暂时不介绍。
- 如果没有索引，我们如何在了一本厚书中找到需要的那一页内容？
- 比如，在读过的小说里找到某个细节。随便翻到一处，查看当前内容，发现目标在此之前；
- 于是翻到之前的任意一处，比对，目标仍然在此之前；重复，直到找到为止。

6.5 数组的应用-数组元素的查找

折半查找（要求数组元素是已排序的，假设是升序排序的）



在下标为left~right的元素中查找值为x的元素下标：

先求出 $mid = (left + right) / 2$ ；若 $x == a[mid]$ ，说明找到；

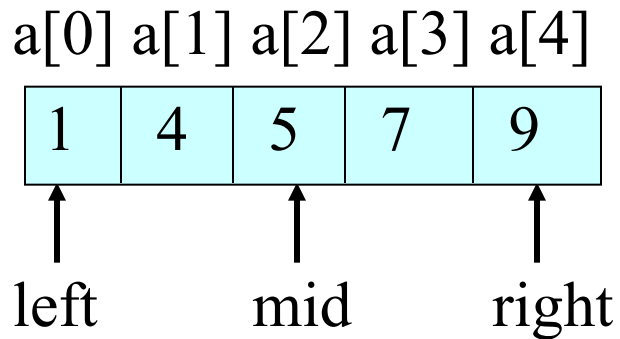
若 $x < a[mid]$ ，说明x可能在下标为left~mid-1的这段元素中，修正查找范围，继续查找。

若 $x > a[mid]$ ，说明x可能在下标为mid+1~right的这段元素中，修正查找范围，继续查找。通过大范围缩减查找范围来提高有序表的查找速度。

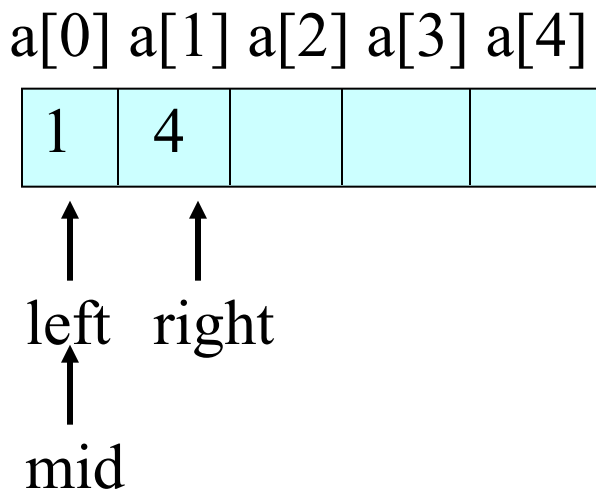


在下标为left~right的元素中查找值为key的元素下标。例如：查找3。 $mid = (left + right) / 2$;

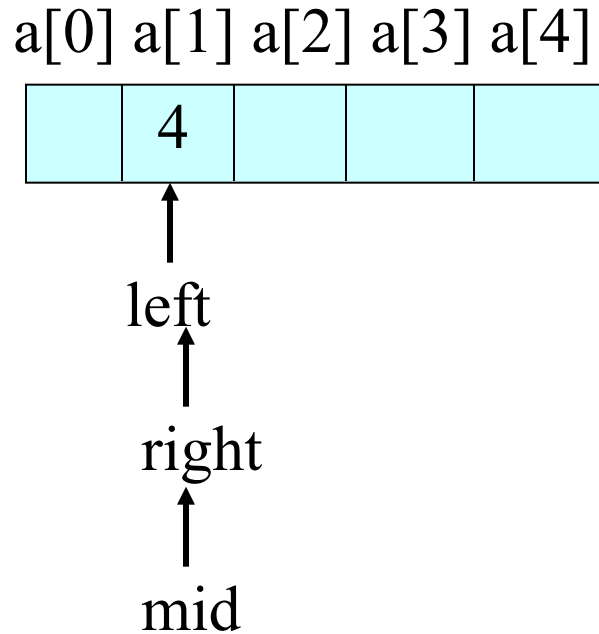
第一趟



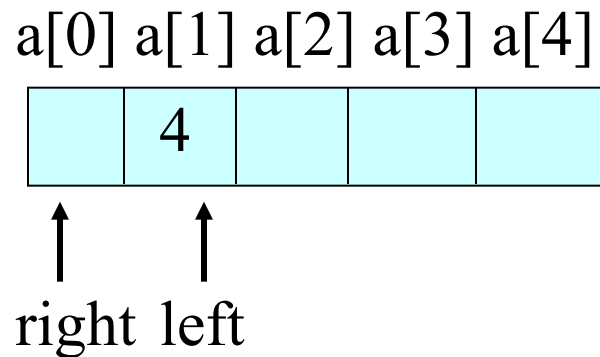
第二趟



第三趟



第四趟



循环结束条件：left>right 或者已经找到

6.5 数组的应用-折半查找

```
int binarySearch(int a[],int n, int x)
{
    int i,j,m;
    for(i=0,j=n-1; i<=j; ){
        m = (i + j)/2;
        if (x == a[m])
            return m;
        else if(x < a[m])
            j = m - 1;
        else
            i = m + 1;
    }
    return -1;
}
```



```
#include <stdio.h>
#define SIZE 10
int binarySearch(int a[],int n, int x);
int main()
{
    int a[SIZE]={1,2,6,7,8,9,12,25,27,36};
    int n,loc,x;

    scanf("%d",&x);
    loc=binarySearch(a,SIZE,x);
    printf("%d  loc=%d\n",x,loc);
    return 0;
}
```

4. 折半查找的性能分析

- 最好的情况: "比较"的次数: 1
- 最坏的情况:
"比较"的次数: $\log_2(n)$
- 时间复杂度 $O(\log_2 n)$

递归算法的二分查找

```
int recBiSearch(int a[],int x,int i,int j)
{ int m;
  if (i<=j){
    m = (i + j)/2;
    if (x == a[m])
      return m;
    else if (x < a[m])
      return recBiSearch(a,x,i,m-1);
    else
      return recBiSearch(a,x,m+1,j);
  }
  else return -1;
}
```



```
#include <stdio.h>
#define SIZE 10
int recBiSearch(int a[],int x,int i,int j);
int main()
{
    int a[SIZE]={1,2,6,7,8,9,12,25,27,36};
    int n,loc,x;

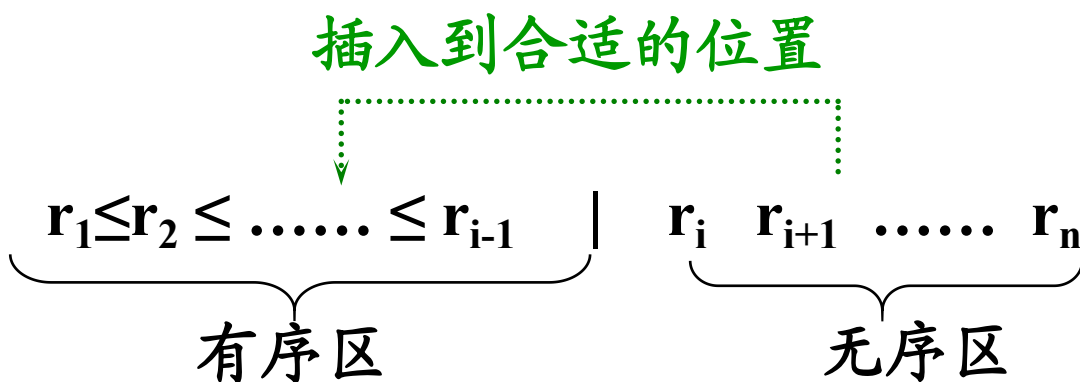
    scanf("%d",&x);
    loc=recBiSearch(a,x,0,SIZE-1);
    printf("%d  loc=%d\n",x,loc);
    return 0;
}
```

6.5 数组的应用-插入排序

有一个已经有序的数据序列，插入一个数，要求插入后此数据序列仍然有序，这种新的排序方法——插入排序法。算法适用于少量数据的排序，时间复杂度为 $O(n^2)$ 。

包括：直接插入排序，折半插入排序。

- **基本操作**：将一个记录插入到已排好序的有序表中，从而得到一个新的、记录数增1的有序表。



插入排序算法伪代码

```
procedure insertion sort( $a_1, a_2, \dots, a_n$ : real numbers with  $n \geq 2$ )  
for  $j := 2$  to  $n$   
     $i := 1$   
    while  $a_j > a_i$   
         $i := i + 1$   
     $m := a_j$   
    for  $k := 0$  to  $j - i - 1$   
         $a_{j-k} := a_{j-k-1}$   
     $a_i := m$   
 $\{a_1, \dots, a_n$  is in increasing order}
```

- 从第2个元素开始依次插入排序

直接插入排序改进:从右开始比较

i



0	1	2	3	4	5	6	7	8
49	38	65	97	76	13	27	<u>49</u>	
38	49							
38	49	65						
38	49	65	97					
38	49	65	76	97				
13	38	49	65	76	97			
13	27	38	49	65	76	97		
<u>13</u>	23	38	49	<u>49</u>	65	76	97	

直接插入排序算法

```
void insertSort(int a[],int n) {  
    int i,j,tmp;  
    for (i=1; i<n; i++){  
        tmp = a[i];  
        for (j=i-1; j>=0 && a[j]>tmp; j--){  
            //若已排序列比之大则后移  
            a[j+1] = a[j];  
        }  
        a[j+1] = tmp; //插入当前元素  
    }  
}
```



```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
void insertSort(int a[],int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    printArray(a,SIZE);
    insertSort(a,SIZE);
    printArray(a,SIZE);
    return 0;
}
```

3. 直接插入排序的性能分析

- 最好的情况（顺序有序）：

"比较"的次数： $\sum_{i=2}^n 1 = n - 1$

"移动"的次数： 0

- 最坏的情况（逆序有序）：

"比较"的次数：

"移动(赋值)"的次数：

$$\sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$$

$$\sum_{i=2}^n (i-1+2) = \frac{(n+4)(n-1)}{2}$$

- 时间复杂度 0 (n^2)

6.5 数组的应用-选择排序

➤ 线性表的基本操作4—数组排序（选择排序）

升序排序算法设计思路：

从 $a[0] \sim a[n-1]$ 元素中找最大元素 $a[\max]$ ， $a[n-1]$ 和 $a[\max]$ 交换；接着，从 $a[0] \sim a[n-2]$ 元素中找最大元素 $a[\max]$ ， $a[n-2]$ 和 $a[\max]$ 交换；依次类推，直到搜索范围只剩 $a[0]$ 元素为止。

程序设计考虑：

利用查找数组最大元素的函数对线性表升序排序

```
void selectSort(int a[],int n)
{ int i,j,max,tmp;
  for (i=1; i<=n-1; i++){
    max = n-i;
    for (j=0; j<=n-1-i; j++){
      if (a[j] > a[max])
        max = j;
    }
    if (n-i != max){
      tmp = a[n-i];
      a[n-i] = a[max];
      a[max] = tmp;
    }
  }
}
```



```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
void selectSort(int a[],int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    printArray(a,SIZE);
    selectSort(a,SIZE);
    printArray(a,SIZE);
    return 0;
}
```

2. 简单选择排序性能分析

- 最好情况

正序序列：比较 $\sum_{i=1}^{n-1} n-i$ 移动 0

- 最坏情况

逆序序列：比较 $\sum_{i=1}^{n-1} n-i$ 移动(赋值) $3*(n-1)$

- 平均情况

时间复杂度 $O(n^2)$ 空间复杂度 $O(1)$

6.5 冒泡排序

- 向池塘丢一颗石子，你会看见气泡从底部慢慢地浮上来，先浮上来的气泡最大。
- 相邻数据元素之间两两比较，若为逆序，则交换位置，上浮一步。

冒牌排序算法思路：

- 1、每一趟过程通过相邻元素两两比较和交换将最大的元素冒泡到最后(下标为0的元素最靠前)；
- 2、需要 $(n-1)$ 趟过程完成数组排序 (n 是元素个数)；



•第一趟比较



•第二趟比较



•第三趟比较



•第四趟比较

➤ 线性表的基本操作4—数组排序（冒泡排序）

```
procedure bubblesort( $a_1, \dots, a_n$  : real numbers with  $n \geq 2$ )  
  for  $i := 1$  to  $n - 1$   
    for  $j := 1$  to  $n - i$   
      if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
  { $a_1, \dots, a_n$  is in increasing order}
```

```
void bubbleSort(int a[],int n){  
    int i,j,tmp;  
    for (i=1;i<=n-1;i++){  
        for (j=0;j<=n-1-i;j++){  
            if (a[j]>a[j+1]){  
                tmp=a[j];  
                a[j]=a[j+1];  
                a[j+1]=tmp;  
            }  
        }  
    }  
}
```




```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
void bubbleSort(int a[],int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    printArray(a,SIZE);
    bubbleSort(a,SIZE);
    printArray(a,SIZE);
    return 0;
}
```

冒泡排序算法改进-没有交换就终止

```
int bubbleSort2(int a[],int n)
{
    int i,j,tmp,swap;
    for(i=1,swap=1; i<=n-1 && swap>0; i++){
        for(j=0,swap=0; j<=n-1-i; j++){
            if(a[j] > a[j+1]){
                tmp = a[j];
                a[j] = a[j+1];
                a[j+1] = tmp;
                swap++;
            }
        }
    }
    return i;//返回排序的趟数
}
```



```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
int bubbleSort2(int a[],int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    printArray(a,SIZE);
    int cnt;
    cnt=bubbleSort2(a,SIZE);
    printArray(a,SIZE);
    printf("cnt=%d\n",cnt);
    return 0;
}
```

用冒泡思想判定数组是否排好序

```
/*judge a[] is up sort?*/  
int isSort(int a[],int n)  
{  
    int i,sorted;  
    for(i=0,sorted=1; i<=n-2 && sorted==1; i++){  
        if(a[i] > a[i+1])  
            sorted=0;  
    }  
  
    return sorted;  
}
```

```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
int bubbleSort2(int a[],int n);
int isSort(int a[],int n);
int main()
{
    int a[SIZE]={2,1,6,9,8,36,27,25,12,7};
    printArray(a,SIZE);
    if (isSort(a,SIZE)==1)
        printf("Sorted\n");
    else printf("Unsort\n");
    bubbleSort2(a,SIZE);
    printArray(a,SIZE);
    if (isSort(a,SIZE)==1)
        printf("Sorted\n");
    else printf("Unsort\n");
    return 0;
}
```

- 最好情况
正序序列

"比较": $n-1$

"移动": 0

- 最坏情况
逆序序列

"比较":

"移动(赋值)":

$$\sum_{i=1}^{n-1} (n - i) = \frac{n(n - 1)}{2}$$

$$3 * \sum_{i=1}^{n-1} (n - i) = 3 * \frac{n(n - 1)}{2}$$

- 平均情况

时间复杂度 $O(n^2)$

6.5 数组的应用4-冒泡排序

练习：

设计冒泡排序算法，实现降序排序。

分析：每一趟过程通过两两比较和交换将较小的元素往后冒泡。

-
- 排序算法有很多种，你平时用哪些方法排序呢？
 - 我们先从一种非常实用并且有趣的排序算法开始。

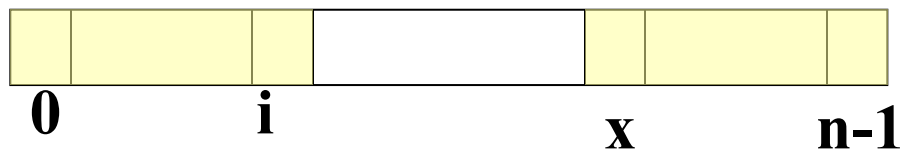
练习2：将一个整数数组逆序存储。

问题分析：

实现逆序的手段是第*i*个元素和倒数第*i*个元素交换。

交换次数： $n/2$ 次（ n 为数组元素个数），故 i 的变化范围是： $0 \sim n/2-1$

$a[i]$ 与 $a[x]$ 交换，则由 $i-0=n-1-x$ ，得 $x=n-1-i$



整数数组逆置函数设计

```
void reverseSeq(int a[], int n)
{ //将整数数组中的元素逆序存放
    int i,tmp;
    for(i=0; i<=(n/2-1); i++){
        tmp = a[i];
        a[i] = a[n-1-i];
        a[n-1-i] = tmp;
    }
}
```

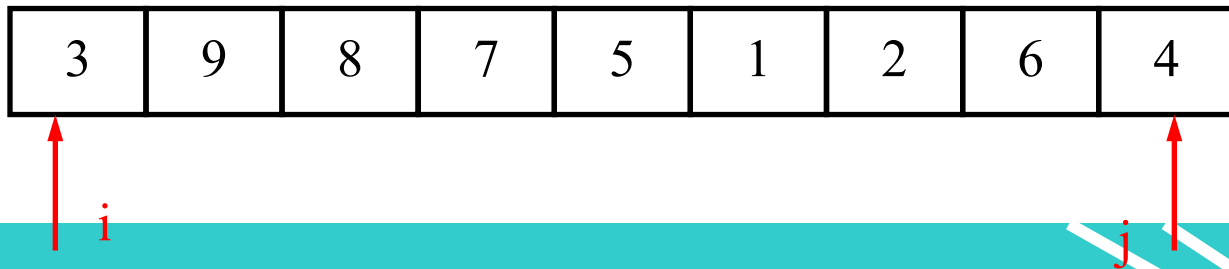


```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
void reverseSeq(int a[], int n);
int main()
{
    int a[SIZE]={1,2,6,7,8,9,12,25,27,36};
    reverseSeq(a,SIZE);
    printArray(a,SIZE);
    return 0;
}
```

将数组划分为奇偶两部分

已知整型数组a，设计算法将其调整为左右两部分，左边所有元素为奇数，右边所有元素为偶数。

```
void oddEvenPartition(int a[], int n){  
    int i=0,j=n-1,tmp;  
    while(i<j){  
        while (i<j && a[i]%2!=0) i++;  
        while (i<j && a[j]%2==0) j--;  
        if (i<j){  
            tmp=a[i]; a[i]=a[j];a[j]=tmp;}  
    }  
}
```





```
#include <stdio.h>
#define SIZE 10
void printArray(const int a[],int n);
void oddEvenPartition(int a[], int n);
int main()
{
    int a[SIZE]={1,2,6,7,8,9,12,25,27,36};
    oddEvenPartition(a,SIZE);
    printArray(a,SIZE);
    return 0;
}
```

回顾：三整数排序问题的标签法

思路是把 a 、 b 、 c 这3个变量本身改成 $a \leq b \leq c$ 的形式。

实现过程：

首先 比较 a 和 b 的大小，

如果 $a > b$ ，则 交换 a 和 b （利用前面讲过的变量交换法）；

接下来 比较 a 和 c 的大小，

如果 $a > c$ ，则 交换 a 和 c ；

最后比较 b 和 c 的大小，

如果 $b > c$ ，则 交换 b 和 c ；

